

Face Detection and Privacy

John Wensink

CSC515 Foundations of Computer Vision

Colorado State University – Global Campus

Dr. Bingdong Li

May, 10, 2025

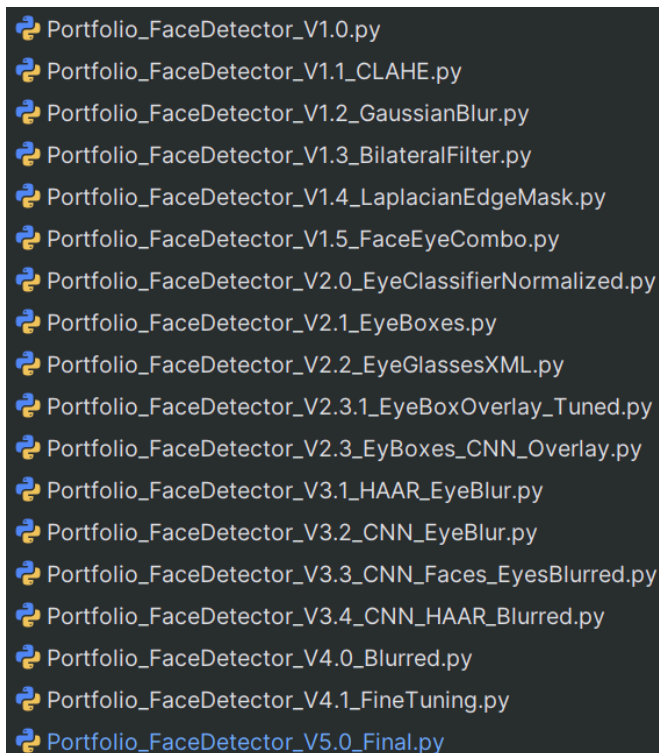
Face Detection and Privacy

For this project, I developed a face detection and anonymization system using OpenCV in Python to showcase the various computer vision techniques covered in this course. The primary objective was to accurately detect human faces in various lighting and environmental conditions and apply targeted blurring to the eyes within each detected face. To meet the assignment requirements, I selected four images from the web that contained meaningful variation in orientation, lighting, and subject count.



The project began with a minimum viable implementation using OpenCV's *haarcascade_frontalface_default.xml* classifier to detect faces in grayscale-converted images (GeeksforGeeks, 2025.) As expected, detection on unprocessed inputs was inconsistent,

especially for images with poor contrast or nonuniform illumination. Image preprocessing became a focus early on, and several preprocessing techniques were attempted. In addition to the mandatory grayscale conversion required in HAAR-based classifiers, Gaussian blur and CLAHE histogram equalization were introduced early to improve detection accuracy. CLAHE helped to enhance local contrast across the non-uniform lighting conditions, while Gaussian blur reduced noise without compromising edge detection. Additional methods, such as bilateral filtering and Laplacian edge masking, were also tested but negatively affected accuracy and pipeline stability.



- Portfolio_FaceDetector_V1.0.py
- Portfolio_FaceDetector_V1.1_CLAHE.py
- Portfolio_FaceDetector_V1.2_GaussianBlur.py
- Portfolio_FaceDetector_V1.3_BilateralFilter.py
- Portfolio_FaceDetector_V1.4_LaplacianEdgeMask.py
- Portfolio_FaceDetector_V1.5_FaceEyeCombo.py
- Portfolio_FaceDetector_V2.0_EyeClassifierNormalized.py
- Portfolio_FaceDetector_V2.1_EyeBoxes.py
- Portfolio_FaceDetector_V2.2_EyeGlassesXML.py
- Portfolio_FaceDetector_V2.3.1_EyeBoxOverlay_Tuned.py
- Portfolio_FaceDetector_V2.3_EyBoxes_CNN_Overlay.py
- Portfolio_FaceDetector_V3.1_HAAR_EyeBlur.py
- Portfolio_FaceDetector_V3.2_CNN_EyeBlur.py
- Portfolio_FaceDetector_V3.3_CNN_Faces_EyesBlurred.py
- Portfolio_FaceDetector_V3.4_CNN_HAAR_Blurred.py
- Portfolio_FaceDetector_V4.0_Blurred.py
- Portfolio_FaceDetector_V4.1_FineTuning.py
- Portfolio_FaceDetector_V5.0_Final.py

Despite improvements in contrast and noise handling, false positives remained a significant challenge, particularly in complex scenes with background textures and sharp edges that mimicked facial structures.



Two strategies were implemented to reduce these detections. First, the HAAR classifier parameters were tuned to require a higher level of agreement across detection scales. The `minNeighbors` parameter was adjusted to a value of 3 to increase the number of consistent detections required before a face bounding box would be accepted. Additionally, a second validation layer was introduced by implementing PyTorch's Multi-Task Cascaded Convolutional Neural Network (MTCNN) model (Nath, 2024) (Timesler, 2020.) MTCNN provided more reliable face bounding boxes and added confidence scores and facial landmarks, allowing for higher-quality filtering later in the pipeline. Only faces detected by MTCNN with a confidence score of .85 or higher were retained for further processing. The combination of a deep learning framework and HAAR cascade classifiers reduced false positives caused by noise or background patterns that could not be tuned using the HAAR parameters alone.

Once faces were reliably detected, the next phase was anonymizing the eyes. Initial designs used OpenCV's *haarcascade_eye.xml* to detect eyes within the previously detected face region. Rectangles were drawn around the eyes, and a strong Gaussian blur was applied using a large kernel size to ensure sufficient blurring. However, the classifier frequently returned overlapping boxes, with multiple detections for a single eye or false-positives triggered by non-eye features such as eyebrows or shadows. I asked an LLM (Grimoire, 2025) for ideas on how to reduce redundant detections, and it recommended using an Intersection over Union (IoU) approach to filter overlapping eye boxes (Rosebrock, 2016.) Following this advice, I implemented a sorting and suppression strategy. All eye bounding boxes were first sorted by area from smallest to largest, under the assumption that the smaller boxes were the more precise encapsulation of the true eye region. Then, using an IoU threshold of 0.3, any new detection that significantly overlapped with an already accepted box was discarded. An additional parameter required that each face retained at most two non-overlapping eye boxes, which immediately and dramatically improved the visual precision and reliability of the eye detections. Despite this improvement, false negatives were still an issue where eyes were occasionally missed, especially in cases where faces were small, distant, or rotated. A final enhancement was implemented using MTCNN's facial landmark output to address these edge cases. MTCNN provides five key landmarks per face, including the eye coordinates (Gradilla, 2020.) After validation of a face detection with at least 85% confidence, the script extracted the eye landmarks from each accepted face. If a landmark fell within a region that the HAAR classifier had already blurred, it was ignored. Otherwise, a square region centered around the eye landmark was defined and blurred using the same Gaussian kernel settings. This additional pass ensured that all eyes were anonymized regardless of orientation, distance, or visibility. A final verification step overlaid a

blue dot on each detected eye to confirm that the system had successfully identified and processed them.



(Red = HAAR face, Blue = MTCNN face, Green = HAAR eyes, Blue Dot = MTCNN eyes)

This hybrid architecture, combining traditional HAAR cascades for initial detections with a deep learning CNN for verification and correction, delivered impressive results across various lighting conditions and facial orientations. The system demonstrated high precision with low false-positive rates, with consistent anonymization across all images. Through this project, I gained practical experience designing and evaluating layered detection pipelines, tuning model parameters, and combining classical and modern computer methods to solve real-world challenges. I developed a deeper understanding of how preprocessing choices directly affected the model's performance and detection reliability. The iterative testing and design process emphasized the importance of modular design and validation when working with variable image data from the real world. This project reinforced the value of combining theoretical knowledge with hands-on experimentation to build an effective computer vision solution.

References

GeeksforGeeks. (2025, April 17). Face Detection using Cascade Classifier using OpenCV.

<https://www.geeksforgeeks.org/face-detection-using-cascade-classifier-using-opencv-python/>

Gradilla, R. (2020, July 27). Multi-task Cascaded Convolutional Networks (MTCNN) for face detection and facial landmark alignment. Medium.

<https://medium.com/@iselagradilla94/multi-task-cascaded-convolutional-networks-mtcnn-for-face-detection-and-facial-landmark-alignment-7c21e8007923>

Grimoire. (2025). *Face and Eye Detector [LLM conversation]*. OpenAI ChatGPT.

<https://chat.openai.com/>

Nath, S. (2024, August 10). *How MTCNN detects faces: A simple guide to powerful technology*.

LinkedIn.

<https://www.linkedin.com/pulse/how-mtcnn-detects-faces-simple-guide-powerful-technology-sujal-nath-lea7c>

Rosebrock, A. (2016, November 7). Intersection over Union (IoU) for object detection.

PyImageSearch.

<https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>

Timesler. (2020). *Guide to MTCNN in facenet-pytorch*. Kaggle.

<https://www.kaggle.com/code/timesler/guide-to-mtcnn-in-facenet-pytorch>

